A Base Safety Theorem

We prove type safety of Base. The language has no mutable cells, so the store never changes and a step is just $e \longrightarrow e'$.

Each binary operator $o \in \{+, -, \wedge, \vee\}$ has a signature $o : \tau_a \times \tau_b \to \tau_o$ (used by T-Op), with $+, - : \mathsf{num} \times \mathsf{num} \to \mathsf{num}$ and $\wedge, \vee : \mathsf{bool} \times \mathsf{bool} \to \mathsf{bool}$.

A.1 Type Safety

Theorem A.1. (Base Type Safety)

1. If $\cdot \vdash e : \tau$ and $e \longrightarrow e'$ then $\cdot \vdash e' : \tau$
2. If $\cdot \vdash e : \tau$ then e is a value or $e \longrightarrow e'$ for some e'.

Proof. By Lemma A.2 and Lemma A.3. □

Lemma A.2. (Base Preservation) If $\Gamma \vdash e_1 : \tau$ and $e_1 \longrightarrow e_2$ then $\Gamma \vdash e_2 : \tau$.

Proof of Lemma A.2.

$e_1 = E[e_1'], e_2 = E[e_2'], e_1' \dashrightarrow e_2'$	`Step`
$\Gamma \vdash e_1' : \tau_1$	Lemma A.5
$\Gamma \vdash e_2' : \tau_1$	Lemma A.6
$\Gamma \vdash E[e_2'] : \tau$	Lemma A.5

□

Lemma A.3. (Base Progress) If $\cdot \vdash e_1 : \tau$ then e_1 is a value $e_1 \longrightarrow e_2$ for some e_2.

Proof of Lemma A.3. By induction on the derivation of $\cdot \vdash e_1 : \tau$.

- Case T-Var. Impossible, the context is empty.
- Case T-App, $e_1 = e_3\, e_4$. By the I.H. e_3 is a value or steps, and then e_4 is a value or steps. If one of them steps then it steps, else if both are values:

$\cdot \vdash e_3 : \tau_2 \to \tau$	`T-App`
$e_3 = \lambda x : \tau_2.e_5$	Lemma A.4
$e_3\, e_4 \longrightarrow e_5[x := e_4]$	`S-App`

- Other cases are trivial: the expression is a value, or it takes a step by the matching rule of $\dashrightarrow$ (using Lemma A.4 for the shape of values).

□

Lemma A.4. (Canonical forms) If $\Gamma \vdash v : \tau$ then:

- If $\tau = \mathsf{num}$ then $v = n$.
- If $\tau = \mathsf{bool}$ then $v = b$.

- If $\tau = \mathsf{unit}$ then $v = \mathsf{unit}$.
- If $\tau = \tau_1 \to \tau_2$ then $v = \lambda x : \tau_1.e$.
- If $\tau = \langle \tau_1, \tau_2 \rangle$ then $v = \langle v_1, v_2 \rangle$ with $\Gamma \vdash v_1 : \tau_1$ and $\Gamma \vdash v_2 : \tau_2$.

Proof. By type rules T-Num, T-Bool, T-Unit, T-Lambda, T-Tuple. □

Lemma A.5. (Decomposition) If $\Gamma \vdash E[e] : \tau$ then there is τ_1 such that $\Gamma \vdash e : \tau_1$, and for every e' with $\Gamma \vdash e' : \tau_1$ we have $\Gamma \vdash E[e'] : \tau$.

Proof. By induction on E.

- Case $E = \square$. Take $\tau_1 = \tau$. The premise is $\Gamma \vdash e : \tau$, and the conclusion follows immediately.
- Case $E = E_0\ e_0$.

$\Gamma \vdash E_0[e]\ e_0 : \tau$	Given
$\Gamma \vdash E_0[e] : \tau_2 \to \tau$	`T-App`
$\Gamma \vdash e_0 : \tau_2$	`T-App`
$\Gamma \vdash e : \tau_1$	I.H.
$\Gamma \vdash E_0[e'] : \tau_2 \to \tau$	I.H.
$\Gamma \vdash E_0[e']\ e_0 : \tau$	`T-App`

- Other cases are trivial: invert the typing rule of the frame, apply the I.H. to E_0 under the same Γ, and apply the same typing rule again.

□

Lemma A.6. (Redex preservation) If $\Gamma \vdash e_1 : \tau$ and $e_1 \dashrightarrow e_2$ then $\Gamma \vdash e_2 : \tau$.

Proof. By cases on $e_1 \dashrightarrow e_2$.

- Case S-App, $(\lambda x : \tau_1.e)\ v \dashrightarrow e[x := v]$.

$\Gamma \vdash \lambda x : \tau_1.e : \tau_1 \to \tau$	`T-App`
$\Gamma \vdash v : \tau_1$	`T-App`
$\Gamma, x : \tau_1 \vdash e : \tau$	`T-Lambda`
$\Gamma \vdash e[x := v] : \tau$	Lemma A.7

- Case S-Fix, $\mu x : \tau.e \dashrightarrow e[x := \mu x : \tau.e]$.

$\Gamma, x : \tau \vdash e : \tau$	`T-Fix`
$\Gamma \vdash \mu x : \tau.e : \tau$	Given
$\Gamma \vdash e[x := \mu x : \tau.e] : \tau$	Lemma A.7

- Other cases are trivial.

□

Lemma A.7. (Substitution) If $\Gamma, x : \tau_1 \vdash e : \tau$ and $\Gamma \vdash e_1 : \tau_1$ then $\Gamma \vdash e[x := e_1] : \tau$.

Proof. By induction on the derivation of $\Gamma, x : \tau_1 \vdash e : \tau$.

- Case T-Var with $e = x$. Then $\Gamma, x : \tau_1 \vdash x : \tau_1$ and $\tau = \tau_1$.

  $\Gamma \vdash e_1 : \tau_1$ — Given
  $x[x := e_1] = e_1$ — Substitution
  $\Gamma \vdash x[x := e_1] : \tau$

- Case T-Var with $e = y \neq x$. Then $\Gamma, x : \tau_1 \vdash y : \tau$

  $y : \tau \in (\Gamma, x : \tau_1)$ — `T-Var`
  $y : \tau \in \Gamma$ — $y \neq x$
  $\Gamma \vdash y[x := e_1] : \tau$ — `T-Var`, $y[x := e_1] = y$

- Case T-Lambda with $e = \lambda y : \tau_2.e_2$, $y \neq x$ and $y \notin \text{fv}(e_1)$.

  $\Gamma, x : \tau_1, y : \tau_2 \vdash e_2 : \tau_3$ — `T-Lambda`
  $\Gamma, y : \tau_2, x : \tau_1 \vdash e_2 : \tau_3$ — Unordered Context
  $\Gamma, y : \tau_2 \vdash e_2[x := e_1] : \tau_3$ — I.H.
  $\Gamma \vdash \lambda y : \tau_2.e_2[x := e_1] : \tau_2 \to \tau_3$ — `T-Lambda`

- Case T-Fix with $e = \mu y : \tau.e_2$. Same as T-Lambda.

- Case T-App with $e = e_2\ e_3$.

  $\Gamma, x : \tau_1 \vdash e_2 : \tau_2 \to \tau$ — `T-App`
  $\Gamma, x : \tau_1 \vdash e_3 : \tau_2$ — `T-App`
  $\Gamma \vdash e_2[x := e_1] : \tau_2 \to \tau$ — I.H.
  $\Gamma \vdash e_3[x := e_1] : \tau_2$ — I.H.
  $\Gamma \vdash (e_2\ e_3)[x := e_1] : \tau$ — `T-App`

- Other cases are trivial: the expression has no variables, or the I.H. applies to every premise and the same rule is used again.

□